ANO
2026

PROJETO MULTIDISCIPLINAR

Nome: Gabriela Diniz Lopes
RU: 4027159
Polo: Itaúna - MG

Prof. Giuliano Lanes de Almeida, Esp.
Profa. Luciane Yanase Hibara Kanashiro, Me.
Prof. Rodrigo da Silva do Nascimento, Me.
Prof. Winston Sen Lun Fung, Me.

SUMÁRIO

1. INTRODUÇÃO 3
2. OBJETIVOS 3
2.1 Objetivo geral 3
2.2 Objetivos específicos 3
3. ANÁLISE E REQUISITOS 4
3.1 Análise do problema 4
3.2 Requisitos funcionais 4
3.3 Requisitos não funcionais 9
4. ARQUITETURA DA SOLUÇÃO 9
4.1 Visão geral da arquitetura 9
4.2 Tecnologias utilizadas 10
4.3 Estrutura do banco de dados 10
4.4 Comunicação entre os componentes 11
5. IMPLEMENTAÇÃO DA SOLUÇÃO 11
5.1 Desenvolvimento do Front-end 11
5.2 Desenvolvimento da API 11
5.3 Integração com o banco de dados 11
5.4 Limitações da implementação 12
6. TESTES DE QUALIDADE DE SOFTWARE 12
6.1 Objetivo dos testes 12
6.2 Testes funcionais 12
6.3 Testes de validação da API 13
6.4 Testes de integração com o banco de dados 17
6.5 Registro das evidências 19
6.6 Limitações dos testes 19
7. IMPLANTAÇÃO E ACESSO À APLICAÇÃO 20
8. CONCLUSÃO 21
9. REFERÊNCIAS 21
10. DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL 22

1. INTRODUÇÃO

O presente trabalho apresenta o desenvolvimento do Raízes App, uma aplicação web proposta para a rede Raízes do Nordeste, com o objetivo de facilitar o processo de realização de pedidos pelos clientes.

A solução foi planejada considerando a necessidade de permitir que o cliente selecione uma unidade da rede, consulte o cardápio correspondente, visualize os produtos disponíveis e organize seu pedido de maneira simples e intuitiva.

O projeto contempla o desenvolvimento de uma interface web responsiva, uma API para comunicação entre a aplicação e o banco de dados, além de testes voltados à verificação do funcionamento e à validação das principais regras implementadas.

A atividade foi desenvolvida com ênfase na trilha de **Qualidade de Software**, priorizando a definição de requisitos, a elaboração de cenários de teste, a identificação de falhas, a aplicação de correções e o registro de evidências dos resultados obtidos.

O trabalho também considera aspectos importantes para uma futura evolução da solução, como segurança, privacidade dos dados, disponibilidade, desempenho e possibilidade de expansão para outras unidades da rede.

2. OBJETIVOS

2.1 Objetivo geral

Desenvolver e avaliar uma versão inicial do Raízes App, voltada à realização de pedidos pela web, aplicando práticas de Qualidade de Software para verificar o funcionamento das principais funcionalidades e identificar possíveis falhas.

2.2 Objetivos específicos

- Permitir o acesso à aplicação web responsiva.
- Possibilitar a seleção de uma unidade da rede Raízes do Nordeste.
- Apresentar o cardápio e os produtos correspondentes à unidade selecionada.
- Permitir a adição de produtos ao pedido e o ajuste das quantidades.
- Calcular o valor total do pedido e realizar sua finalização.
- Integrar a aplicação web à API e ao banco de dados SQL Server para registrar pedidos e seus itens.

- Definir e executar testes positivos e negativos para verificar as regras implementadas.
- Registrar evidências dos testes e documentar limitações e possibilidades de evolução da solução.

3. ANÁLISE E REQUISITOS

3.1 Análise do problema

A rede Raízes do Nordeste necessita de uma solução tecnológica que facilite o acesso dos clientes ao cardápio e à realização de pedidos. Considerando que a rede possui diferentes unidades, é importante que o cliente consiga selecionar o estabelecimento desejado e visualizar os produtos correspondentes àquela unidade.

O Raízes App foi desenvolvido como uma aplicação web responsiva, com foco na consulta de produtos e na organização e finalização de pedidos. A solução utiliza uma interface web integrada a uma API e a um banco de dados, permitindo o registro dos pedidos e de seus respectivos itens.

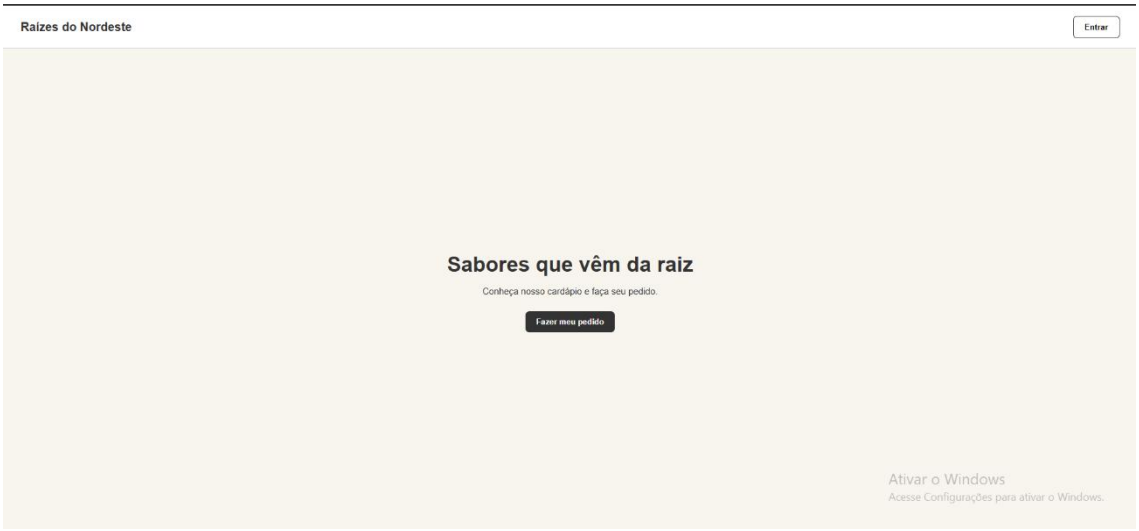
O desenvolvimento priorizou as funcionalidades essenciais para demonstrar o fluxo de realização de pedidos. Funcionalidades adicionais, como controle de estoque, programa de fidelização e integração com serviços externos de pagamento, são consideradas possibilidades de evolução da solução.

3.2 Requisitos funcionais

- **RF001** — Acesso à aplicação: O sistema deve permitir que o cliente acesse a aplicação web responsiva.



RF001 - Acesso à aplicação. Home em resolução mobile.



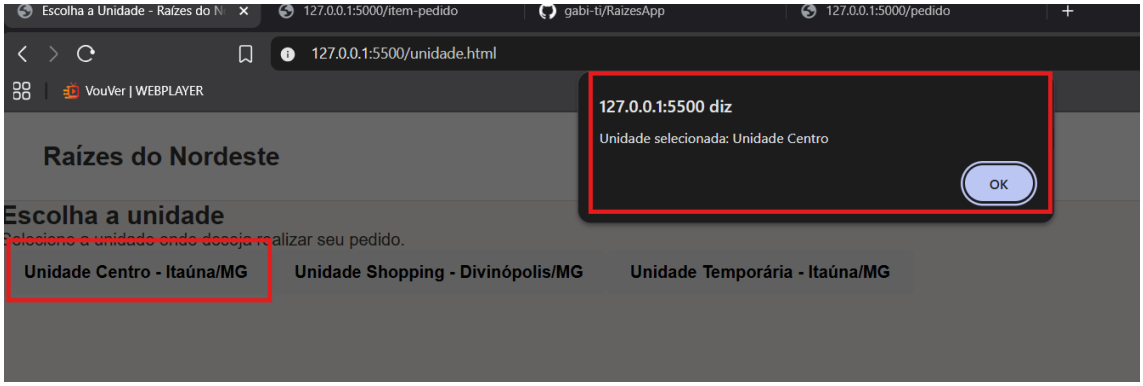
RF001 - Acesso à aplicação. Home em tela Desktop.

- **RF002** — Realizar pedido: O sistema deve permitir que o cliente inicie o processo de realização de um pedido.



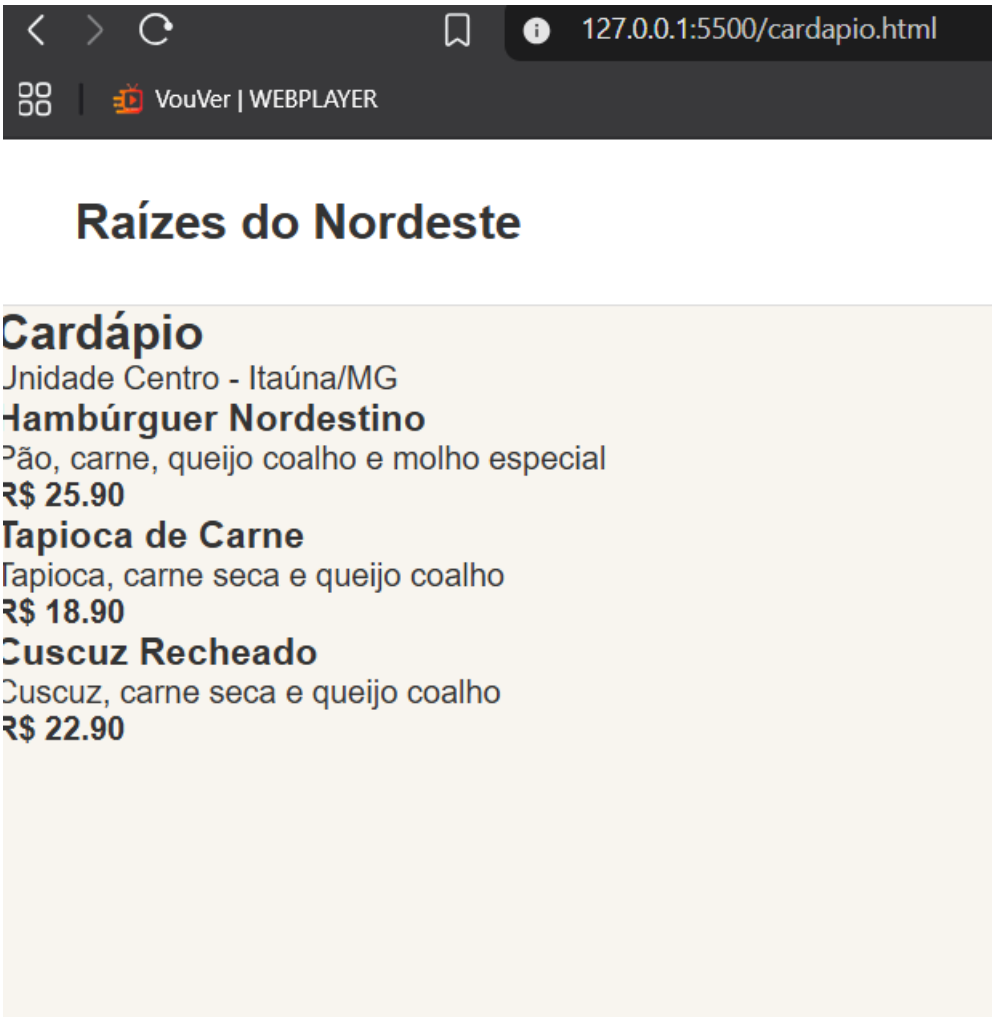
RF002 - Realizar pedido_Botão_FinalizarPedido.

RF003 — Selecionar unidade: O sistema deve permitir que o cliente escolha a unidade da Raízes do Nordeste onde deseja realizar o pedido.



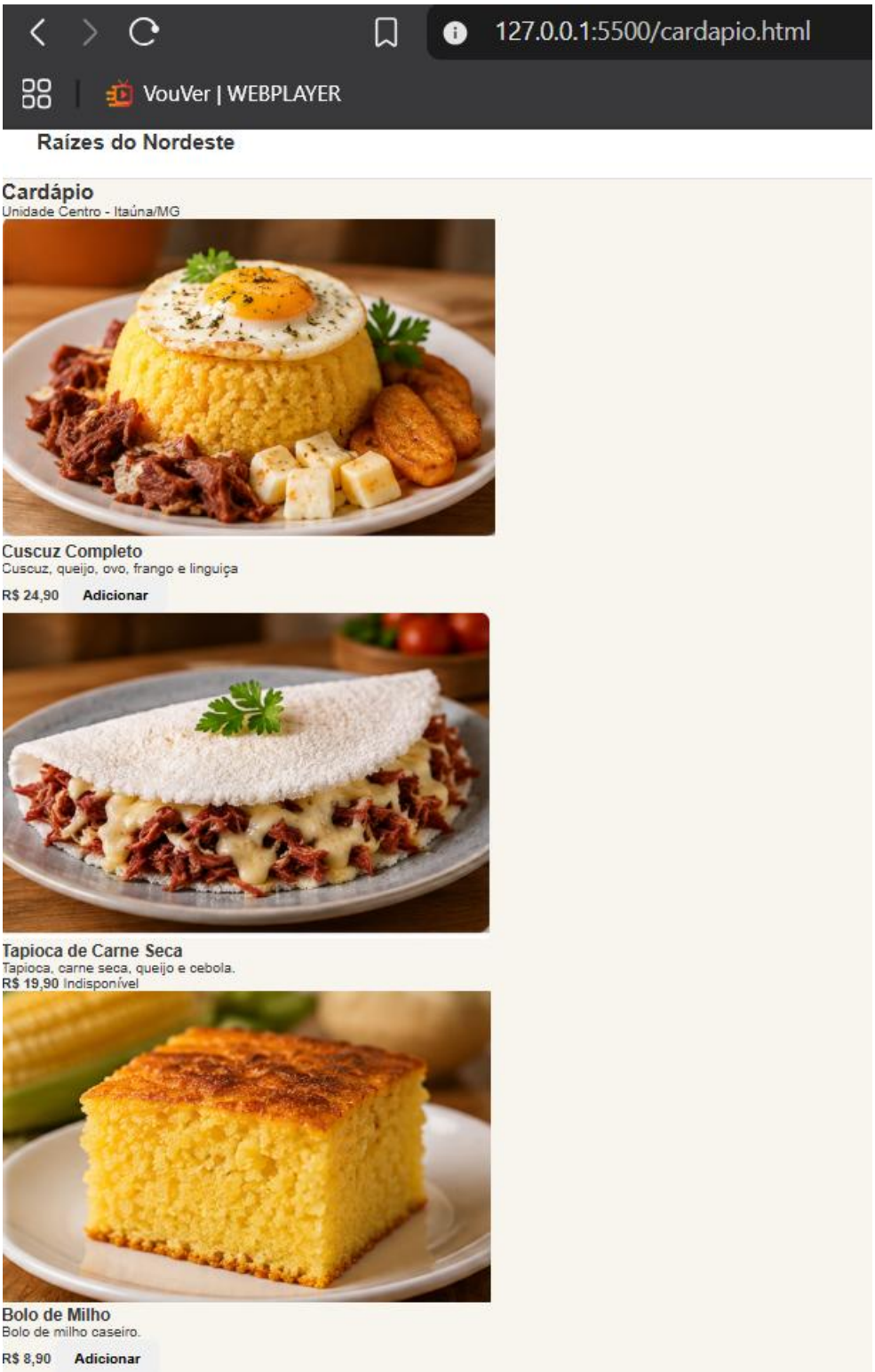
RF003 - Selecionar unidade.

- **RF004** — Visualizar cardápio: O sistema deve apresentar ao cliente o cardápio correspondente à unidade selecionada.



RF004 - Visualizar cardápio.

- **RF005** — Visualizar produtos: O sistema deve apresentar informações dos produtos disponíveis, incluindo imagem, nome, ingredientes resumidos e preço.



RF005 - Visualizar produtos.

- **RF006** — Adicionar produto ao pedido: O sistema deve permitir que o cliente adicione produtos ao pedido.



RF006 - Adicionar produto ao pedido.

3.3 Requisitos não funcionais

- **RNF001** — Responsividade: A interface deve se adaptar a diferentes tamanhos de tela, permitindo o acesso por computadores e dispositivos móveis.
- **RNF002** — Usabilidade: A aplicação deve apresentar navegação clara e intuitiva para facilitar a seleção da unidade, a consulta ao cardápio e a realização do pedido.
- **RNF003** — Integridade dos dados: Os pedidos e seus respectivos itens devem respeitar os relacionamentos definidos no banco de dados.
- **RNF004** — Validação de dados: A API deve validar os dados recebidos, verificando os campos obrigatórios e impedindo o registro de quantidades iguais ou inferiores a zero e preços inválidos.
- **RNF005** — Segurança e privacidade: Uma implantação em ambiente de produção deve considerar medidas de proteção dos dados, controle de acesso e atendimento à Lei Geral de Proteção de Dados Pessoais (LGPD).
- **RNF006** — Disponibilidade e desempenho: A solução deve considerar requisitos de disponibilidade e desempenho para suportar as solicitações dos usuários, inclusive em períodos de maior movimento.

4. ARQUITETURA DA SOLUÇÃO

4.1 Visão geral da arquitetura

- O Raízes App utiliza uma arquitetura dividida em três componentes principais: Front-end, API e banco de dados. Essa organização permite separar a interface apresentada ao usuário das regras de processamento e do armazenamento das informações.
- O **Front-end** foi desenvolvido com HTML, CSS e JavaScript, sendo responsável pela apresentação das telas e pela interação com o cliente durante o processo de realização do pedido.
- A **API** foi desenvolvida em Python, utilizando o framework Flask. Sua função é receber as requisições realizadas pela aplicação web, validar os dados e realizar as operações necessárias no banco de dados.
- O **banco de dados** utiliza o Microsoft SQL Server e armazena informações relacionadas a usuários, unidades, produtos, pedidos e itens dos pedidos.

4.2 Tecnologias utilizadas

- **HTML:** estruturação das páginas da aplicação.
- **CSS:** estilização e organização visual das telas.
- **JavaScript:** implementação das interações e comunicação com a API.
- **Python e Flask:** desenvolvimento da API.
- **Flask-CORS:** configuração do acesso entre a aplicação web e a API.
- **PyODBC:** comunicação entre a API e o banco de dados.
- **Microsoft SQL Server:** armazenamento das informações.
- **Git e GitHub:** versionamento e disponibilização do código-fonte.
- **GitHub Pages:** publicação da interface web para acesso pela internet.

4.3 Estrutura do banco de dados

O banco de dados utilizado no projeto possui cinco tabelas principais:

- **Usuario:** armazena os dados dos usuários.
- **Unidade:** armazena as informações das unidades da rede.
- **Produto:** registra os produtos, seus preços, ingredientes, imagens e disponibilidade.
- **Pedido:** armazena os pedidos realizados, incluindo usuário, unidade, data, status e valor total.
- **ItemPedido:** registra os produtos que compõem cada pedido, suas quantidades e preços unitários.

A estrutura foi organizada para relacionar os pedidos aos usuários e às unidades, bem como os itens aos respectivos pedidos e produtos.

4.4 Comunicação entre os componentes

- Durante a execução local, o Front-end envia requisições à API Flask. A API processa as solicitações, realiza as validações necessárias e acessa o banco de dados SQL Server para consultar ou registrar informações.
- No fluxo de realização de pedidos, a aplicação envia os dados do pedido e, em seguida, os dados de seus itens. Após o processamento, a interface apresenta uma mensagem informando o resultado da operação.
- A publicação do Front-end no GitHub Pages permite visualizar a interface pela internet. Entretanto, a API e o banco de dados permanecem configurados para execução local, de modo que a operação completa do sistema depende da execução desses componentes no ambiente de desenvolvimento.

5. IMPLEMENTAÇÃO DA SOLUÇÃO

5.1 Desenvolvimento do Front-end

- O Front-end do Raízes App foi desenvolvido utilizando HTML, CSS e JavaScript. A interface permite que o cliente acesse a página inicial, selecione uma unidade, consulte o cardápio, visualize os produtos e organize seu pedido.
- A aplicação foi estruturada para facilitar a navegação e permitir o acesso por diferentes tamanhos de tela.

5.2 Desenvolvimento da API

- A API foi desenvolvida em Python, utilizando o framework Flask, e é responsável por intermediar a comunicação entre o Front-end e o banco de dados.
- Foram implementadas rotas para consultar unidades e produtos, registrar pedidos e consultar ou registrar itens dos pedidos. A API também realiza validações para verificar campos obrigatórios, existência dos registros relacionados e consistência das quantidades e dos preços informados.

5.3 Integração com o banco de dados

- O banco de dados Microsoft SQL Server foi utilizado para armazenar as informações do sistema. A integração com a API permite registrar os pedidos e seus respectivos itens, mantendo os relacionamentos entre as tabelas.

- Durante os testes locais, foi possível validar a comunicação entre os componentes e confirmar o registro das informações no banco de dados.

5.4 Limitações da implementação

- A versão desenvolvida contempla as funcionalidades principais do fluxo de realização de pedidos. Entretanto, a aplicação ainda possui limitações, como a ausência de uma estrutura de autenticação completa e a necessidade de disponibilizar a API e o banco de dados em um ambiente acessível externamente para permitir o funcionamento integrado pela internet.
- Essas limitações foram consideradas na avaliação da solução e poderão ser tratadas em futuras evoluções do projeto.

6. TESTES DE QUALIDADE DE SOFTWARE

6.1 Objetivo dos testes

- Os testes foram realizados com o objetivo de verificar o funcionamento das principais funcionalidades do Raízes App, avaliar as validações implementadas na API e identificar possíveis falhas durante o processamento dos pedidos.
- A estratégia contemplou cenários positivos, para verificar operações válidas, e cenários negativos, para avaliar o comportamento do sistema diante de dados inválidos ou informações obrigatórias ausentes.

6.2 Testes funcionais

- Os testes funcionais foram direcionados às principais operações da aplicação, incluindo a consulta de unidades e produtos, o registro de pedidos e a inclusão de itens.
- Também foram avaliadas as validações dos campos obrigatórios, a existência dos registros relacionados e as regras aplicadas às quantidades e aos preços dos produtos.

- Além dos testes funcionais realizados, foram considerados aspectos não funcionais relacionados à usabilidade, segurança, privacidade, disponibilidade e desempenho da aplicação. Entretanto, não foram concluídos testes abrangentes nesses aspectos, nem testes automatizados ou de carga. Essas verificações permanecem como atividades futuras para uma avaliação mais ampla da solução em ambiente de produção.

6.3 Testes de validação da API

- Durante os testes da API, foram verificadas situações como ausência de campos obrigatórios, utilização de usuário, pedido ou produto inexistente, quantidade igual a zero e preço negativo.
- As validações implementadas permitem rejeitar solicitações que não atendem às regras definidas, contribuindo para a consistência das informações armazenadas no banco de dados.

```
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API> Invoke-RestMethod `
>>   -Uri "http://127.0.0.1:5000/pedido" `
>>   -Method POST `
>>   -ContentType "application/json" `
>>   -Body '{"id_unidade":1,"status":"Em aberto","valor_total":25.90}'
Invoke-RestMethod : {
  "erro": "O campo id_usuario \u00e9 obrigat\u00f3rio."
}
No linha:1 caractere:1
+ Invoke-RestMethod `
+ ~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API>
```

CT002 – Dado Obrigatório.

```
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API> Invoke-RestMethod `
>>   -Uri "http://127.0.0.1:5000/pedido" `
>>   -Method POST `
>>   -ContentType "application/json" `
>>   -Body '{"id_usuario":1,"status":"Em aberto","valor_total":25.90}'
Invoke-RestMethod : {
  "erro": "O campo id_unidade \u00e9 obrigat\u00f3rio."
}
No linha:1 caractere:1
+ Invoke-RestMethod `
+ ~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
```

CT002 – IdUnidade Obrigatório.

```
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API> Invoke-RestMethod `
>>   -Uri "http://127.0.0.1:5000/pedido" `
>>   -Method POST `
>>   -ContentType "application/json" `
>>   -Body '{"id_usuario":1,"id_unidade":1,"valor_total":25.90}'
Invoke-RestMethod : {
  "erro": "O campo status \u00e9 obrigat\u00f3rio."
}
No linha:1 caractere:1
+ Invoke-RestMethod `
+ ~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API>
```

CT002 - Status Obrigatório.

```
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API> Invoke-RestMethod `
>>   -Uri "http://127.0.0.1:5000/pedido" `
>>   -Method POST `
>>   -ContentType "application/json" `
>>   -Body '{"id_usuario":1,"id_unidade":1,"status":"Em aberto"}'
Invoke-RestMethod : {
  "erro": "O campo valor_total \u00e9 obrigat\u00f3rio."
}
No linha:1 caractere:1
+ Invoke-RestMethod `
+ ~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API>
```

CT002 – Valor Total Obrigatório.

```
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API> Invoke-RestMethod `
>>   -Uri "http://127.0.0.1:5000/item-pedido" `
>>   -Method POST `
>>   -ContentType "application/json" `
>>   -Body '{"id_pedido":3,"id_produto":1,"quantidade":0,"preco_unitario":25.90}'

id_item mensagem
------- --------
      4 Item do pedido criado com sucesso!

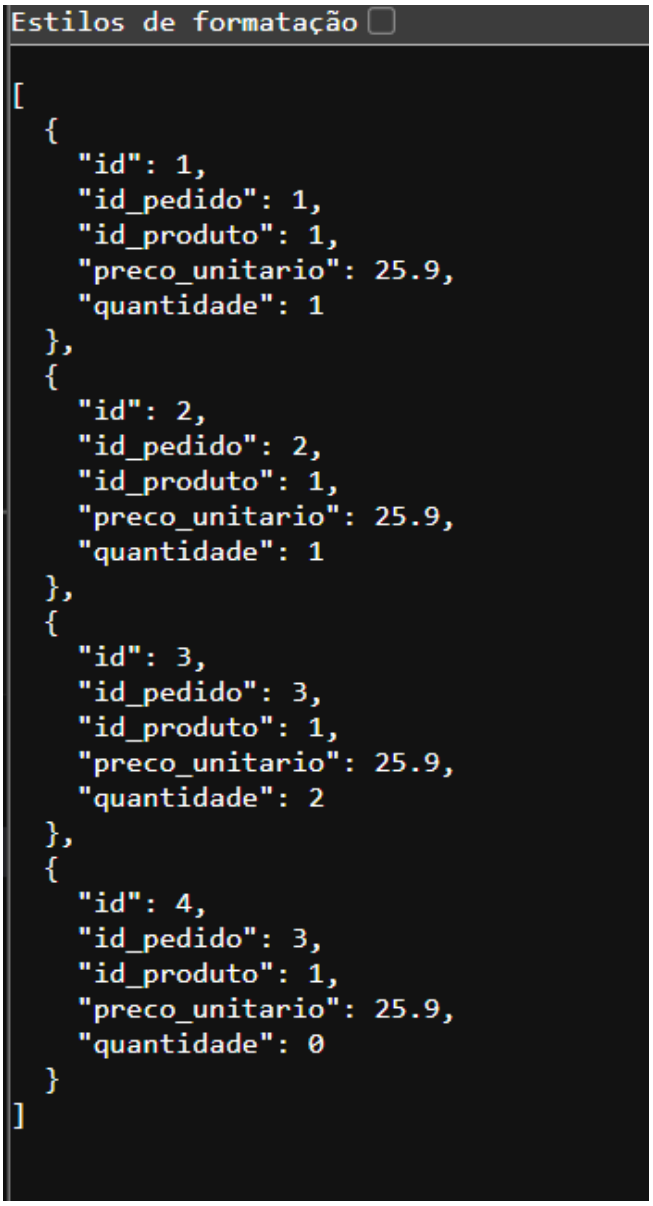

(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API>
```

CT005 – Quantidade zero antes da correção: inclusão indevida de item.

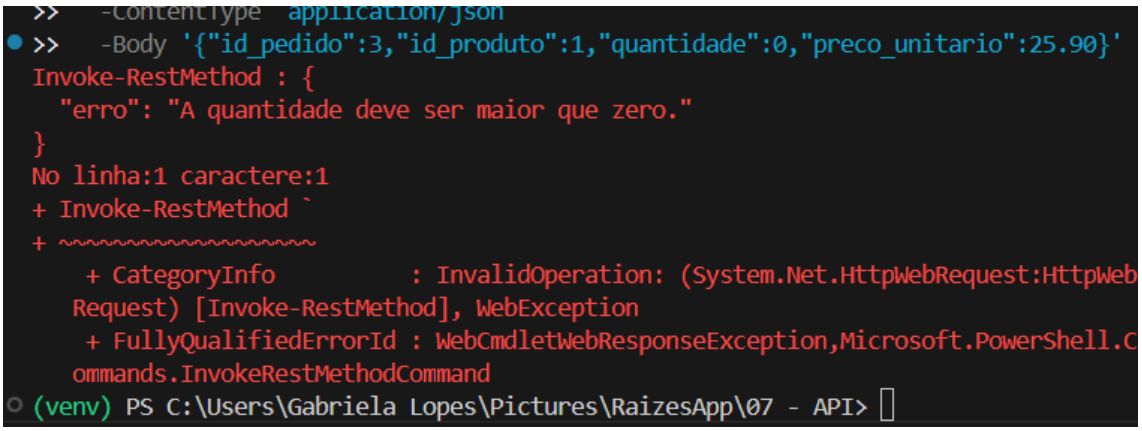
```
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API> Invoke-RestMethod `
>>   -Uri "http://127.0.0.1:5000/item-pedido" `
>>   -Method POST `
>>   -ContentType "application/json" `
>>   -Body '{"id_pedido":3,"id_produto":1,"quantidade":2,"preco_unitario":25.9
0}'
id_item mensagem
------- --------
      3 Item do pedido criado com sucesso!


(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API>
```

CT005 - Validação de quantidade - Teste Positivo.



CT005 - Quantidade Zero Antes Correção.



CT005 - Quantidade Zero Após Correção. Erro

```
PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API>
PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API> & "c:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API\venv\Scripts\Activate.ps1"
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API> Invoke-RestMethod `
>>   -Uri "http://127.0.0.1:5000/item-pedido" `
>>   -Method POST `
>>   -ContentType "application/json" `
>>   -Body '{"id_pedido":3,"id_produto":1,"quantidade":1,"preco_unitario":-10.00}'

id_item mensagem
------- --------
      5 Item do pedido criado com sucesso!


(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API>
```

CT006 - Preço Negativo Antes Correção.

```
  },
  {
    "id": 5,
    "id_pedido": 3,
    "id_produto": 1,
    "preco_unitario": -10.0,
    "quantidade": 1
  }
]
```

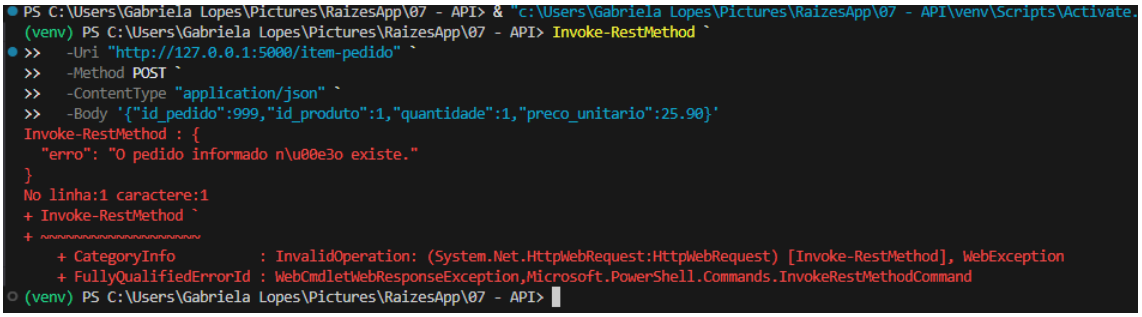
CT006 - Preço Negativo Banco Antes Correção.

```
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API> Invoke-RestMethod `
>>   -Uri "http://127.0.0.1:5000/item-pedido" `
>>   -Method POST `
>>   -ContentType "application/json" `
>>   -Body '{"id_pedido":3,"id_produto":1,"quantidade":1,"preco_unitario":-10.00}'
Invoke-RestMethod : {
  "erro": "O pre\u00e7o deve ser maior que zero."
}
No linha:1 caractere:1
+ Invoke-RestMethod `
+ ~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API>
```

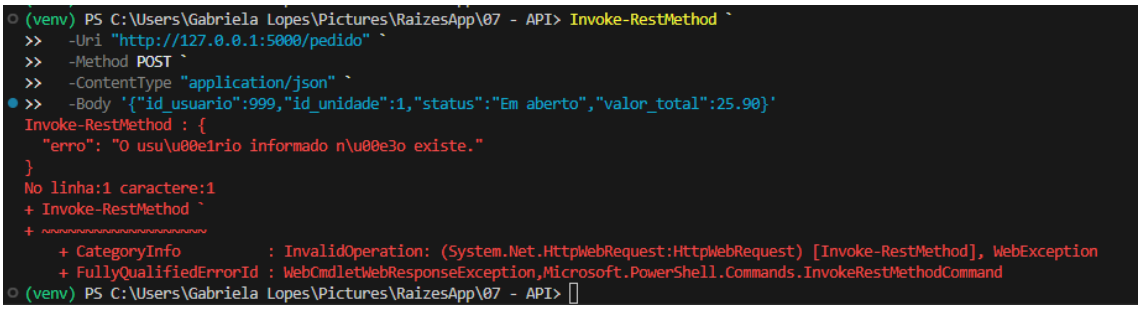
CT006 - Preço Negativo Após Correção.

```
PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API> & "c:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API\venv\Scripts\Activat
ps1"
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API> Invoke-RestMethod `
>>   -Uri "http://127.0.0.1:5000/item-pedido" `
>>   -Method POST `
>>   -ContentType "application/json" `
>>   -Body '{"id_pedido":3,"id_produto":999,"quantidade":1,"preco_unitario":25.90}'
Invoke-RestMethod : {
  "erro": "O produto informado n\u00e3o existe."
}
No linha:1 caractere:1
+ Invoke-RestMethod `
+ ~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
(venv) PS C:\Users\Gabriela Lopes\Pictures\RaizesApp\07 - API>
```

CT007- Produto Inexistente.



CT007 - Pedido Inexistente.



CT007 - Usuário Inexistente.

6.4 Testes de integração com o banco de dados

- Foi realizado um teste de integração para verificar o registro de um pedido e de seus respectivos itens no banco de dados SQL Server.
- No cenário positivo, foi registrado um pedido contendo dois itens de quantidade unitária do produto Cuscuz Completo, totalizando duas unidades, com preço unitário de R$ 24,90 e valor total de R$ 49,80. A consulta ao banco de dados confirmou o registro do pedido e de seu item correspondente.
- Também foi realizado um cenário negativo de finalização de pedido vazio. A aplicação apresentou uma mensagem solicitando a inclusão de pelo menos um produto, e a consulta ao banco de dados confirmou que um novo pedido não foi registrado nesse cenário.

```
12
13  SELECT TOP 10 *
14    FROM Pedido
15    ORDER BY AutoID DESC;
```

100 %

Resultados | Mensagens

	AutoID	IdUsuario	IdUnidade	DataPedido	Status	ValorTotal
1	6	1	1	2026-10-10 21:14:46.783	Em aberto	49.80
2	5	1	1	2026-09-29 18:48:20.373	Em aberto	25.90
3	3	1	1	2026-09-23 19:09:24.580	Em aberto	25.90
4	2	1	1	2026-09-20 18:24:36.530	Em aberto	25.90
5	1	1	1	2026-09-19 17:23:00.610	Em aberto	25.90

Fluxo_Pedido_Gravado_Banco.

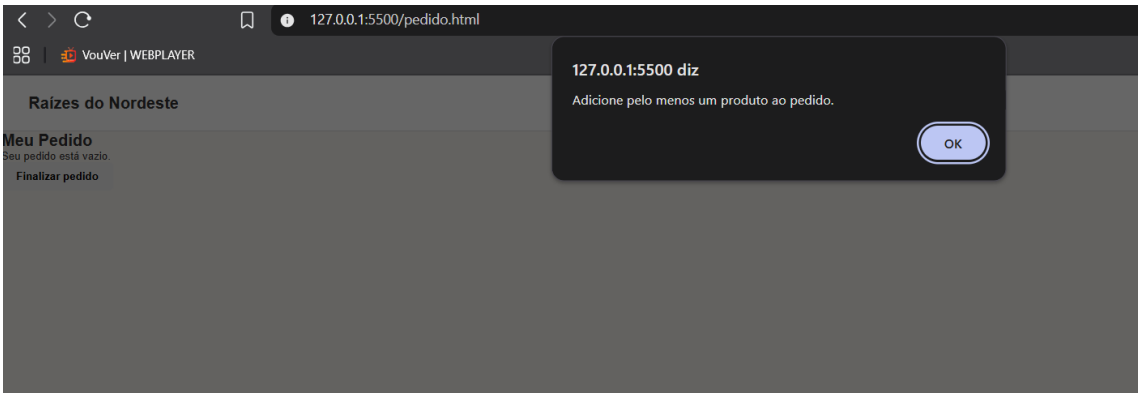
```
16
17  SELECT TOP 10 *
18    FROM ItemPedido
19    ORDER BY AutoID DESC;
```

100 %

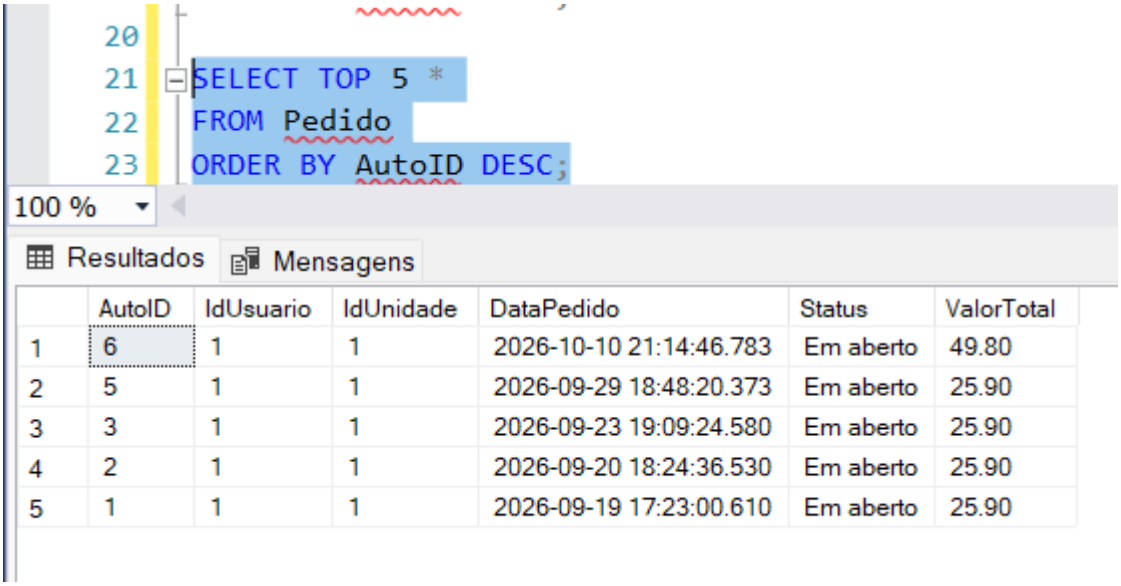
Resultados | Mensagens

	AutoID	IdPedido	IdProduto	Quantidade	PrecoUnitario
1	9	6	1	2	24.90
2	8	3	1	1	25.90
3	3	3	1	2	25.90
4	2	2	1	1	25.90
5	1	1	1	1	25.90

Fluxo_ItemPedido_Gravado_Banco.



Fluxo_Pedido_Vazio_Bloqueado.



	AutoID	IdUsuario	IdUnidade	DataPedido	Status	ValorTotal
1	6	1	1	2026-10-10 21:14:46.783	Em aberto	49.80
2	5	1	1	2026-09-29 18:48:20.373	Em aberto	25.90
3	3	1	1	2026-09-23 19:09:24.580	Em aberto	25.90
4	2	1	1	2026-09-20 18:24:36.530	Em aberto	25.90
5	1	1	1	2026-09-19 17:23:00.610	Em aberto	25.90

Fluxo_Pedido_Vazio_Nao_Gravado_Banco.

6.5 Registro das evidências

As evidências dos testes foram registradas por meio de capturas de tela da aplicação, dos resultados das requisições à API e de consultas ao banco de dados. Os registros demonstram os cenários avaliados, as falhas identificadas e as correções aplicadas durante a validação das funcionalidades implementadas.

6.6 Limitações dos testes

- Os testes realizados concentraram-se nas funcionalidades implementadas e nos cenários validados durante o desenvolvimento local. Testes abrangentes de desempenho, disponibilidade, segurança e comportamento sob grande volume de acessos não foram considerados concluídos apenas com essas verificações.
- Esses aspectos permanecem como pontos importantes para uma futura validação em ambiente de produção.

7. IMPLANTAÇÃO E ACESSO À APLICAÇÃO

A aplicação Raízes do Nordeste foi desenvolvida uma arquitetura composta por Front-end, API e banco de dados, permitindo a separação das responsabilidades entre as diferentes camadas do sistema.

Acesso ao front-end:
https://gabi-ti.github.io/raizes_do_nordeste/08%20-%20Front-end/index.html

Repositório do projeto:
https://github.com/gabi-ti/raizes_do_nordeste

Protótipo da interface no Figma:
https://www.figma.com/design/IYvtIDxdkUGh2Z8MHHiNyK/Raizes-App?node-id=0-1&t=lOXEI8KXYVUQJKHe-1

https://www.figma.com/proto/IYvtIDxdkUGh2Z8MHHiNyK/Raizes-App?node-id=1-4&t=wVVuSvS0p9m5PDtj-0&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=1%3A4

O protótipo foi elaborado no Figma para apoiar o planejamento visual da aplicação e a organização das telas e dos fluxos de navegação do Raízes App.

O Front-end foi publicado por meio do GitHub Pages, permitindo o acesso à interface pela internet. O código-fonte do projeto está disponível no repositório público do GitHub, juntamente com os documentos e artefatos desenvolvidos.

A API foi implementada em Python, utilizando Flask, e o banco de dados foi desenvolvido em Microsoft SQL Server. Durante o desenvolvimento e os testes, esses componentes foram executados localmente no computador utilizado para a implementação do projeto.

É importante destacar que a publicação do Front-end não disponibiliza automaticamente a API e o banco de dados na internet. Dessa forma, quando o site é acessado por outro computador ou dispositivo, a aplicação não consegue se comunicar com a API executada apenas no ambiente local de desenvolvimento. Para que todas as funcionalidades operem de forma integrada pela internet, será necessário hospedar a API em um servidor acessível externamente e disponibilizar o banco de dados em uma infraestrutura compatível.

Apesar dessa limitação, a integração entre o Front-end, a API e o banco de dados foi testada localmente. Os resultados e as evidências dos testes realizados estão apresentados nesta documentação.

Assim, o projeto apresenta a implementação das principais funcionalidades do Raízes App, seus artefatos técnicos e os resultados obtidos durante a validação, além de documentar as etapas necessárias para sua futura implantação em ambiente de produção.

8. CONCLUSÃO

O desenvolvimento do Raízes App permitiu aplicar conhecimentos relacionados à análise de requisitos, ao desenvolvimento de aplicações web, à integração com banco de dados e à Qualidade de Software.

Durante o projeto, foram implementadas as principais funcionalidades do fluxo de pedidos, incluindo a consulta de unidades e produtos, o registro de pedidos e a inclusão de seus respectivos itens. Os testes realizados permitiram verificar cenários válidos e inválidos, identificar falhas e validar as correções aplicadas.

A integração entre o Front-end, a API Flask e o banco de dados SQL Server foi validada em ambiente local. Entretanto, a aplicação ainda apresenta limitações, como a ausência de autenticação completa e a necessidade de disponibilizar a API e o banco de dados em um ambiente externo para permitir o funcionamento integrado pela internet.

Conclui-se que o projeto atingiu seu objetivo de desenvolver uma versão inicial do Raízes App e aplicar práticas de Qualidade de Software na validação das funcionalidades implementadas. Como possibilidades de evolução, destacam-se a ampliação dos testes, a melhoria dos mecanismos de segurança, a implantação em ambiente de produção e a implementação de funcionalidades adicionais.

9. REFERÊNCIAS

FLASK. Flask Documentation. Disponível em: https://flask.palletsprojects.com/. Acesso em: 06 set. 2026.

FIGMA. Figma Help Center. Disponível em: https://help.figma.com/. Acesso em: 25 ago. 2026.

GITHUB. GitHub Docs. Disponível em: https://docs.github.com/. Acesso em: 12 out. 2026.

MICROSOFT. SQL Server Documentation. Disponível em: https://learn.microsoft.com/sql/sql-server/. Acesso em: 07 set. 2026.

PYTHON SOFTWARE FOUNDATION. Python Documentation. Disponível em: https://docs.python.org/3/. Acesso em: 06 set. 2026.

10. DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Durante o desenvolvimento deste trabalho, foi utilizada a ferramenta ChatGPT, da OpenAI, como apoio ao processo de elaboração e revisão da documentação.

A ferramenta foi consultada para auxiliar na organização e revisão de textos, esclarecer dúvidas conceituais e apoiar a análise da estrutura do trabalho, dos requisitos e da documentação relacionada ao projeto Raízes App.

Exemplos de prompts utilizados:

- “Ajude-me a revisar e organizar a documentação do projeto.”
- “Ajude-me a melhorar a clareza e a organização dos textos.”

Os trechos aproveitados foram revisados e ajustados durante o desenvolvimento do trabalho. A implementação da aplicação, os testes realizados, as evidências obtidas e as decisões sobre o projeto devem ser descritos de acordo com o processo efetivamente executado pela estudante.

A utilização da ferramenta é declarada em conformidade com as orientações da atividade, cabendo à estudante a responsabilidade pela veracidade das informações apresentadas e pela versão final entregue.